

第六章 迭代器模式、观察者模式与策略模式

一、单选题

1. 迭代器模式又被称为()模式（单选题）

- A. 代理
- B. 命令
- C. 发布订阅
- D. 游标

答案：D

解析：迭代器模式（Iterator Pattern）又被称为游标模式（Cursor Pattern）。迭代器提供了一种统一的方法来顺序访问聚合对象中的各个元素，就像数据库中的游标（Cursor）一样可以逐条遍历记录。选项 A 代理模式用于控制对对象的访问，是结构型模式；选项 B 命令模式用于将请求封装为对象，是行为型模式；选项 C 发布订阅模式是观察者模式的别名，用于实现一对多的状态通知。

2. JDK 中提供观察者模式支持的类是()（单选题）

- A. Iterator、Collection
- B. Observable、Observer
- C. Command、Invoker
- D. Strategy、Context

答案：B

解析：JDK 在 `java.util` 包中提供了对观察者模式的内置支持：`Observable` 类作为被观察的目标（Subject），提供了添加观察者、删除观察者、通知观察者等方法；`Observer` 接口作为观察者，定义了 `update(Observable o, Object arg)` 方法用于接收目标发送的通知。需要注意的是，`Observable` 是类而非接口，这在某些场景下有局限性。选项 A 中 `Iterator` 和 `Collection` 属于 Java 集合框架，是迭代器模

式的体现；选项 C 中 Command 和 Invoker 是命令模式的核心角色；选项 D 中 Strategy 和 Context 是策略模式的核心角色。

3. 策略模式中负责调用算法的角色是()（单选题）

- A. 环境类 Context
- B. 客户端
- C. 具体策略类
- D. 抽象策略类

答案：A

解析：策略模式包含三个核心角色：环境类 Context 持有一个抽象策略对象的引用，负责在需要时调用策略对象的算法方法，它是客户端和策略之间的桥梁；抽象策略类声明了所有具体策略类必须实现的算法接口；具体策略类实现了抽象策略类定义的算法接口，封装了具体的算法逻辑。Context 将算法的选择与算法的实现分离，使得算法可以独立于使用它的客户端而变化。选项 B 客户端负责创建和配置策略对象；选项 C 具体策略类是被调用的对象；选项 D 抽象策略类只定义接口不负责调用。

4. 迭代器模式中负责创建迭代器对象的是()（单选题）

- A. 抽象迭代器
- B. 具体迭代器
- C. 客户端
- D. 抽象聚合类

答案：D

解析：在迭代器模式中，聚合类（Aggregate）负责创建迭代器对象。抽象聚合类定义了一个创建迭代器的抽象方法（如 createIterator() 或 iterator()），具体聚合类实现该方法，返回一个与自身匹配的具体迭代器实例。这是工厂方法模式在迭代器模式中的体现——聚合类充当“工厂”，负责生产与之配对的迭代器。选项 A

抽象迭代器只定义遍历接口不创建对象；选项 B 具体迭代器是被创建的对象而非创建者；选项 C 客户端调用聚合类的创建方法来获取迭代器，但并不直接创建迭代器。

5. Java 内置迭代器所在包是 ()（单选题）

- A. java.lang
- B. java.net
- C. java.util
- D. java.io

答案：C

解析：Java 内置的 `Iterator` 接口定义在 `java.util` 包中。`java.util` 包是 Java 集合框架的核心包，其中定义了 `Iterator` 接口（提供 `hasNext()`、`next()`、`remove()` 等方法）、`Iterable` 接口（提供 `iterator()` 方法使集合支持 `for-each` 循环）、以及各种集合类如 `ArrayList`、`LinkedList`、`HashSet`、`HashMap` 等。选项 A `java.lang` 是 Java 核心语言包（自动导入，含 `String`、`Object` 等）；选项 B `java.net` 是网络编程包；选项 D `java.io` 是输入输出编程包。

6. 策略模式的核心是 ()（单选题）

- A. 封装一系列算法并可相互替换
- B. 控制真实对象访问
- C. 一对多状态联动通知
- D. 遍历聚合对象不暴露内部结构

答案：A

解析：策略模式的核心思想是定义一系列算法，将每一个算法封装到独立的策略类中，并使它们可以相互替换。通过统一的策略接口，客户端可以在运行时根据需要选择不同的算法，而无需修改使用算法的上下文代码。选项 B 控制真实对象访问是代理模式的核心作用；选项 C 一对多状态联动通知是观察者模式的核心特征；选项

D 遍历聚合对象不暴露内部结构是迭代器模式的核心作用。策略模式让算法的变化独立于使用算法的客户端。

7. 策略模式又称为()模式（单选题）

- A. 政策
- B. 游标
- C. 事务
- D. 发布订阅

答案：A

解析：策略模式（Strategy Pattern）又被称为政策模式（Policy Pattern）。 “Strategy”（策略）和“Policy”（政策）在含义上非常接近——都表示根据不同情况采取不同方案的思维。在软件设计中，策略模式允许在运行时根据条件选择不同的算法或行为方案，就像政策会随着情况变化而调整一样。选项 B 游标是迭代器模式的别名；选项 C 事务是命令模式的别名；选项 D 发布订阅是观察者模式的别名。

8. 以下不属于策略模式角色的是()（单选题）

- A. 具体观察者
- B. 具体策略类
- C. 抽象策略类
- D. 环境类 Context

答案：A

解析：策略模式包含三个核心角色：抽象策略类定义算法接口；具体策略类实现具体的算法；环境类 Context 持有策略引用并负责调用算法。选项 A“具体观察者”是观察者模式中的角色，负责接收观察目标发出的通知并作出响应，与策略模式无关。本题的关键是区分行为型模式之间的角色差异——观察者模式关注状态变化通知，策略模式关注算法的封装与切换。

9. 影院售票系统不同折扣方案适合使用哪种设计模式() (单选题)

- A. 观察者模式
- B. 迭代器模式
- C. 代理模式
- D. 策略模式

答案: D

解析: 影院售票系统有多种折扣方案(如学生折扣、会员折扣、团购折扣、节假日折扣等), 每种折扣方案对应一种计算票价的具体算法。策略模式非常适合这种场景: 将每种折扣算法封装成独立的策略类(如 StudentDiscountStrategy、VIPDiscountStrategy 等), 环境类(如 TicketContext) 持有当前使用的折扣策略, 在计算票价时调用策略对象的算法方法。这样新增一种折扣只需新增一个策略类, 无需修改原有代码, 符合开闭原则。选项 A 观察者模式适合股票价格通知等场景; 选项 B 迭代器模式适合遍历集合元素; 选项 C 代理模式适合控制对象访问。

10. 迭代器模式属于() (单选题)

- A. 创建型模式
- B. 对象行为型模式
- C. 模板模式
- D. 结构型模式

答案: B

解析: 迭代器模式属于对象行为型设计模式。行为型模式关注对象之间的职责分配和协作方式。迭代器模式将聚合对象的遍历行为分离到独立的迭代器对象中, 通过迭代器来控制遍历的过程和顺序, 这属于对“行为”的封装和组织。特别注意: 迭代器模式不属于结构型模式(结构型关注对象组合如代理、适配器)、不属于创建型模式(创建型关注对象创建如工厂、单例)。选项 C 模板模式是另一个具体的行为型模式名, 不是分类类别。

11. MVC 架构中，对应观察者模式观察目标的是()（单选题）

- A. Controller
- B. 都不是
- C. Model
- D. View

答案：C

解析：在 MVC（Model-View-Controller）架构中，Model（模型）对应观察者模式中的观察目标角色。Model 负责管理应用程序的数据和业务逻辑，当数据发生变化时，Model 会通知所有注册的 View（视图）更新显示。View 相当于观察者，当它收到 Model 的通知后，会从 Model 获取最新数据进行界面刷新。Controller 负责处理用户输入并更新 Model，不直接参与观察者模式的角色对应。这种 Model-View 的观察关系是 MVC 架构实现数据与显示分离的核心机制。

12. 增加新的聚合类时，迭代器模式通常需要()（单选题）

- A. 修改原有抽象迭代器
- B. 无需任何改动
- C. 修改原有聚合类代码
- D. 同步增加新的迭代器类

答案：D

解析：在迭代器模式中，每个聚合类通常都有与之配套的迭代器类来完成特定的遍历逻辑。当新增一种聚合类时（如从 ArrayList 新增到 TreeSet），由于新聚合类的内部存储结构不同（如数组、链表、树、哈希表），遍历方式也不同，一般需要同步实现一个新的具体迭代器类来适配新聚合类的遍历需求。选项 A 错误，抽象迭代器接口不需要修改；选项 B 错误，需要新增迭代器类；选项 C 错误，原有聚合类代码无需修改。这也是迭代器模式的一个缺点——每增加一种聚合类通常都要增加对应的迭代器类。

13. 以下哪项不是策略模式的适用场景() (单选题)

- A. 替代多重 if-else 条件语句
- B. 一对多对象状态联动
- C. 封装算法保证保密性
- D. 动态选择多种算法

答案: B

解析: 策略模式的适用场景包括: 替代复杂的 if-else 或 switch 条件分支, 将每个分支的逻辑抽取为独立的策略类; 封装算法以保证算法实现的独立性和保密性; 在运行时根据条件动态选择和切换算法。选项 B“一对多对象状态联动”是观察者模式的适用场景——当一个对象的状态发生变化时, 所有依赖它的对象都能自动收到通知并更新。区分这两个模式的关键在于: 策略模式解决“多选一”(选择一种算法执行), 观察者模式解决“一变多”(一个变化通知多个依赖)。

14. 策略模式属于() (单选题)

- A. 对象行为型模式
- B. 创建型模式
- C. 结构型模式
- D. 代理模式

答案: A

解析: 策略模式属于对象行为型设计模式。行为型模式关注对象之间的交互和职责分配, 策略模式通过将算法封装到独立的策略对象中, 让算法的选择和使用方式在运行时动态变化, 这属于对象行为的范畴。策略模式不是创建型模式(不关注对象创建)、不是结构型模式(不关注对象之间的静态结构组合)。选项 D 代理模式是一个具体的结构型模式, 不是分类名称。

15. 以下不属于迭代器模式角色的是() (单选题)

- A. 具体目标类

- B. 抽象聚合类
- C. 具体聚合类
- D. 抽象迭代器

答案：A

解析：迭代器模式包含四个核心角色：抽象聚合类定义创建迭代器的接口；具体聚合类实现创建迭代器的方法并返回具体迭代器实例；抽象迭代器定义遍历元素所需的接口（如 `hasNext()`、`next()`）；具体迭代器实现具体遍历逻辑并维护当前遍历位置。选项 A “具体目标类”是观察者模式中的角色，它存储观察者列表并在状态变化时通知所有观察者，与迭代器模式无关。

16. 下列不属于观察者模式别名的是()（单选题）

- A. 源-监听器模式
- B. 游标模式
- C. 模型-视图模式
- D. 发布-订阅模式

答案：B

解析：观察者模式有多个常用别名：发布-订阅模式（Publish-Subscribe）因为观察目标发布通知、观察者订阅通知；源-监听器模式（Source-Listener）因为事件源产生事件、监听器响应事件；模型-视图模式（Model-View）因为模型数据变化、视图随之更新；从属者模式（Dependents）因为观察者依赖于观察目标的状态。选项 B “游标模式”是迭代器模式的别名，因为迭代器像数据库游标一样逐条遍历数据，与观察者模式无关。

17. 观察者模式中状态发生改变的对象称为()（单选题）

- A. 策略
- B. 观察目标
- C. 观察者

D. 迭代器

答案：B

解析：在观察者模式中，被观察的对象称为观察目标（Subject），也叫做被观察者（Observable）。观察目标是状态变化的发起方——当它的状态发生改变时，它会主动遍历自己的观察者列表，调用每个观察者的更新方法（update），将变化通知给所有依赖它的观察者。观察者（Observer）是状态变化的接收方，它对观察目标的变化作出响应。选项 A 策略是策略模式的概念；选项 D 迭代器是迭代器模式的概念。

18. 迭代器模式的核心作用是()（单选题）

- A. 顺序访问聚合对象元素且不暴露内部结构
- B. 封装请求支持撤销
- C. 控制对象访问权限
- D. 动态替换算法

答案：A

解析：迭代器模式的核心作用是提供一种方法，使得可以顺序访问聚合对象（如集合、列表、树等）中的各个元素，同时不需要暴露聚合对象的底层存储结构（如数组、链表、哈希表、二叉树等）。客户端通过统一的迭代器接口（如 `hasNext()` 和 `next()`）就可以遍历任何类型的集合，无需关心集合内部是用什么数据结构实现的。选项 B 封装请求支持撤销是命令模式的核心作用；选项 C 控制对象访问权限是代理模式（保护代理）的核心作用；选项 D 动态替换算法是策略模式的核心作用。

19. 观察者模式定义的对象关系是()（单选题）

- A. 一对一
- B. 一对多
- C. 多对多
- D. 无固定关系

答案：B

解析：观察者模式定义了对象之间的一种一对多依赖关系。在这种关系中，“一”指的是一个观察目标对象（Subject），“多”指的是多个观察者对象（Observer）。当这个目标对象的状态发生变化时，所有依赖它的观察者对象都会自动收到通知并做出相应更新。例如：一个股票价格数据源（一）可以被多个显示面板（多）——K线图、分时图、涨跌幅榜同时观察，当股价变化时所有面板同时更新。

20. 观察者模式属于()（单选题）

- A. 工厂模式
- B. 对象行为型模式
- C. 结构型模式
- D. 创建型模式

答案：B

解析：观察者模式属于对象行为型设计模式。它描述的是对象之间的一种交互行为：一个对象（观察目标）的状态变化会触发对其他对象（观察者）的通知行为。这种“状态改变→自动通知→响应更新”的联动机制体现了对象之间的行为协作关系，因此归类为行为型模式。选项A工厂模式是一个具体的创建型模式；选项C结构型模式关注对象的静态组合；选项D创建型模式关注对象的创建过程。

二、多选题

21. 迭代器模式的优点包括()（多选题）

- A. 符合开闭原则，扩展方便
- B. 必须暴露聚合对象内部结构
- C. 支持多种方式遍历同一聚合对象
- D. 不会增加系统类数量
- E. 简化聚合类设计

答案：ACE

解析：迭代器模式的优点包括：选项 A 符合开闭原则——新增聚合类和迭代器类无需修改已有代码，扩展方便；选项 C 支持对同一聚合对象进行多种遍历方式——如正序遍历、倒序遍历、跳跃遍历等，每种遍历方式对应一个不同的迭代器实现；选项 E 简化聚合类设计——将遍历逻辑从聚合类中分离到迭代器类中，聚合类只需关注数据存储。选项 B 错误，迭代器模式恰恰是为了“不暴露”聚合对象的内部结构；选项 D 错误，迭代器模式会增加系统中类的数量（每个聚合类通常都需要配套的迭代器类）。

22. 策略模式的适用场景有()（多选题）

- A. 需封装算法保护数据结构隐私
- B. 一对多对象状态自动更新
- C. 系统需要动态切换多种算法
- D. 遍历集合不暴露内部结构
- E. 需要消除复杂的多重条件判断

答案：ACE

解析：策略模式的适用场景包括：选项 A 当需要将算法封装起来，既保护算法内部的实现隐私，又对外提供统一的调用接口时；选项 C 当系统需要在运行时根据条件动态选择或切换不同算法时（如根据用户等级切换折扣算法）；选项 E 当系统中存在大量 if-else 或 switch 条件语句来选择不同行为时，可以用策略模式消除这些复杂分支。选项 B 一对多状态自动更新是观察者模式的适用场景；选项 D 遍历集合不暴露内部结构是迭代器模式的适用场景。

23. 迭代器模式包含的角色有()（多选题）

- A. 具体聚合类
- B. 具体迭代器
- C. 抽象迭代器

- D. 抽象聚合类
- E. 具体目标类

答案：ABCD

解析：迭代器模式包含四大核心角色：抽象聚合类定义创建迭代器的接口；具体聚合类实现创建迭代器的方法，返回与该聚合类匹配的具体迭代器实例；抽象迭代器定义遍历元素的统一接口（如 `first()`、`next()`、`hasNext()`、`currentItem()` 等）；具体迭代器实现具体的遍历算法，维护遍历过程中的当前位置。选项 E“具体目标类”是观察者模式中的角色，负责维护观察者列表和发送通知，不属于迭代器模式。

24. 策略模式的优点包括()（多选题）

- A. 完美支持开闭原则
- B. 可替换继承关系
- C. 客户端无需了解任何策略类
- D. 便于算法复用
- E. 避免多重条件选择语句

答案：ABDE

解析：策略模式的优点包括：选项 A 完美支持开闭原则——新增算法只需新增策略类，无需修改 `Context` 和已有策略类的代码；选项 B 可替换继承关系——用组合（`Context` 持有策略对象引用）替代继承，避免了继承带来的类层次膨胀和灵活性不足；选项 D 便于算法复用——策略类是独立的，可以在不同的 `Context` 中复用相同的算法；选项 E 避免多重条件选择语句——将 `if-else` 分支逻辑分散到各个策略类中。选项 C 错误，客户端通常需要知道有哪些具体策略类可用，才能选择并创建合适的策略对象传入 `Context`。

25. 观察者模式的优点有()（多选题）

- A. 目标变化可自动通知所有观察者
- B. 符合开闭原则

- C. 循环依赖不会产生问题
- D. 实现表示层与逻辑层分离
- E. 支持广播通信

答案：ABDE

解析：观察者模式的优点包括：选项 A 观察目标状态变化时自动通知所有已注册的观察者，无需手动逐个调用；选项 B 符合开闭原则——新增观察者只需新增观察者类并注册到目标即可，无需修改目标和已有观察者；选项 D 可以实现表示层（View）和数据逻辑层（Model）的分离，是 MVC 架构的基础；选项 E 支持广播通信——一个目标可以同时向多个观察者发送通知。选项 C 错误，观察者模式中如果出现循环依赖（A 观察 B 的同时 B 也观察 A），会导致无限循环通知，可能引发栈溢出或系统崩溃。

26. 策略模式包含的角色有()（多选题）

- A. 抽象策略类
- B. 抽象目标类
- C. 环境类 Context
- D. 具体迭代器
- E. 具体策略类

答案：ACE

解析：策略模式包含三个核心角色：选项 A 抽象策略类定义所有具体策略类必须实现的算法接口，通常是一个接口或抽象类；选项 C 环境类 Context 持有抽象策略对象的引用，负责调用策略对象的算法方法，它在客户端和策略之间充当中介；选项 E 具体策略类实现了抽象策略类中的算法接口，每个具体策略类封装了一种具体的算法实现。选项 B“抽象目标类”是观察者模式的角色；选项 D“具体迭代器”是迭代器模式的角色。

27. 观察者模式包含的角色有()（多选题）

- A. 抽象目标
- B. 抽象迭代器
- C. 具体目标
- D. 抽象观察者
- E. 具体观察者

答案：ACDE

解析：观察者模式包含四个核心角色：选项 A 抽象目标（Subject）定义添加、删除和通知观察者的接口；选项 C 具体目标（ConcreteSubject）实现抽象目标的接口，维护具体的状态数据，当状态改变时通知所有观察者；选项 D 抽象观察者（Observer）定义观察者的更新接口（如 update 方法）；选项 E 具体观察者（ConcreteObserver）实现更新接口，在收到目标通知后做出具体的响应。选项 B“抽象迭代器”属于迭代器模式，与观察者模式无关。

28. 下列属于行为型设计模式的有()（多选题）

- A. 代理模式
- B. 策略模式
- C. 迭代器模式
- D. 观察者模式
- E. 命令模式

答案：BCDE

解析：行为型设计模式关注对象之间的通信和职责分配。选项 B 策略模式属于行为型（处理算法的选择和使用）；选项 C 迭代器模式属于行为型（处理聚合对象的遍历行为）；选项 D 观察者模式属于行为型（处理对象间的通知联动行为）；选项 E 命令模式属于行为型（处理请求的封装与执行行为）。选项 A 代理模式属于结构型模式，它关注的是对象之间的静态代理结构关系，而不是行为交互。

29. 关于迭代器、观察者、策略模式说法正确的有()（多选题）

- A. 三者都完全不遵循开闭原则
- B. 三者都属于对象行为型模式
- C. 策略模式用于算法封装与替换
- D. 观察者用于一对多状态联动
- E. 迭代器用于集合遍历解耦

答案：BCDE

解析：选项 B 正确，迭代器、观察者、策略三种模式都属于对象行为型设计模式（GoF 分类）；选项 C 正确，策略模式的核心职能就是算法封装与动态替换；选项 D 正确，观察者模式的核心职能就是实现对象间一对多的状态联动通知；选项 E 正确，迭代器模式的核心职能就是聚合对象的遍历而不暴露内部结构。选项 A 错误，三种模式都遵循开闭原则——都可以在不修改原有代码的情况下通过新增类来扩展系统功能。

30. 下列属于观察者模式别名的有()（多选题）

- A. 从属者模式
- B. 源-监听器模式
- C. 游标模式
- D. 模型-视图模式
- E. 发布-订阅模式

答案：ABDE

解析：观察者模式有多个别名：选项 A 从属者模式强调观察者依赖于观察目标的状态；选项 B 源-监听器模式强调事件源产生事件、监听器处理事件；选项 D 模型-视图模式强调数据模型变化驱动视图更新，是 MVC 架构的基础；选项 E 发布-订阅模式强调目标发布消息、观察者订阅消息的通信机制。选项 C“游标模式”是迭代器模式的别名，因为迭代器像数据库游标一样逐个遍历元素，不属于观察者模式的别名。

三、判断题

31. 迭代器模式增加新聚合类时，无需新增迭代器类。()（判断题）

A. 对

B. 错

答案：B

解析：此题说法错误（选 B）。在迭代器模式中，聚合类和迭代器类是配对出现的。当增加一种新的聚合类时（例如新实现一个基于树结构的自定义集合类），通常需要同步实现一个对应的新迭代器类来处理该聚合类的遍历逻辑。因为不同的聚合类内部存储结构不同（数组、链表、树、哈希表等），遍历算法也不同，需要独立的迭代器实现。这是迭代器模式增加新聚合类时的一个必要步骤，也是该模式增加系统类数量的缺点体现。

32. 策略模式无法避免程序中多重 if-else 条件判断。()（判断题）

A. 对

B. 错

答案：B

解析：此题说法错误（选 B）。策略模式的一个核心使用场景恰恰是消除程序中复杂的多重 if-else 条件判断。在传统写法中，根据不同条件执行不同逻辑时往往使用大量的 if-else 或 switch 语句，导致代码臃肿且难以扩展。策略模式将每种条件分支对应的逻辑封装到一个独立的策略类中，客户端只需从上下文获取策略并调用即可，从根本上避免了长串的条件分支。这就是策略模式常说的“用组合替代条件判断”的含义。

33. 迭代器模式可以在不暴露聚合对象内部结构的前提下遍历元素。()（判断题）

A. 对

B. 错

答案：A

解析：此题说法正确（选 A）。这是迭代器模式最核心的特征和设计目标。聚合对象（如集合、列表、树等）的内部存储结构（底层是用数组、链表、哈希表还是二叉树实现）对客户端完全隐藏，客户端只需要通过迭代器提供的统一接口（如 `hasNext()` 和 `next()`）就能完成遍历操作。这种封装性使得聚合类的内部实现可以自由变化而不会影响使用它的客户端代码，是“单一职责原则”和“信息隐藏”思想的体现。

34. 策略模式中客户端不需要知道任何具体策略类。（）（判断题）

A. 对

B. 错

答案：B

解析：此题说法错误（选 B）。在策略模式的标准用法中，客户端通常需要知道具体策略类的存在。因为客户端负责根据当前条件创建或选择合适的具体策略对象，并将其传递给环境类 `Context`。例如在折扣系统中，客户端需要根据用户类型创建 `StudentDiscountStrategy` 或 `VIPDiscountStrategy`，然后传给 `TicketContext`。不过，可以结合简单工厂模式或依赖注入来进一步降低客户端对具体策略类的直接依赖。

35. Java 事件处理机制底层采用了观察者模式思想。（）（判断题）

A. 对

B. 错

答案：A

解析：此题说法正确（选 A）。Java 的 GUI 事件处理机制是观察者模式的典型应用。在 Java Swing/JavaFX 中，按钮（`JButton`）等组件相当于观察目标，当用户点击按钮时（状态变化），按钮会通知所有已注册的事件监听器（`ActionListener`），调用其 `actionPerformed` 方法。事件源（如按钮、文本框）是观察目标，事件监听

器是观察者，事件对象（ActionEvent）携带状态变化的具体信息。这种“事件源-监听器”的模式本质上就是观察者模式。

36. 策略模式可以将多种算法封装并实现相互替换。()（判断题）

A. 对

B. 错

答案：A

解析：此题说法正确（选 A）。这是策略模式的核心定义和设计目标。策略模式定义了一系列算法，将每个算法封装在独立的策略类中，并让它们实现统一的策略接口，从而使得这些算法可以在运行时相互替换。例如在排序场景中，可以定义 BubbleSortStrategy、QuickSortStrategy、MergeSortStrategy 等，它们都实现 SortStrategy 接口，客户端可以根据数据规模动态选择合适的排序算法，而调用排序的代码无需改变。

37. 观察者模式是一对一依赖关系的设计模式。()（判断题）

A. 对

B. 错

答案：B

解析：此题说法错误（选 B）。观察者模式定义的是一对多依赖关系，而非一对一。一个观察目标对象可以被多个观察者对象同时观察，当目标的状态发生变化时，所有注册的观察者都会收到通知。例如：一个天气数据站（观察目标）可以同时被手机 App、网站、电子显示屏等多个终端（观察者）观察。如果是一对一关系，观察者模式的优势（广播通知、一对多自动更新）就无从体现了。

38. 影院多类型会员打折场景适合使用策略模式开发。()（判断题）

A. 对

B. 错

答案：A

解析：此题说法正确（选A）。影院多类型会员（如普通会员、银卡会员、金卡会员、钻石会员）对应不同的折扣算法，这是策略模式的典型应用场景。每种会员类型可以封装为一个独立的策略类，实现统一的折扣计算接口。当会员类型切换时，只需要替换 Context 中的策略对象即可，无需修改计算票价代码。这种设计使得新增会员类型时只需新增一个策略类，完全符合开闭原则。

39. MVC 架构中 Model 相当于观察者模式的观察目标。（）（判断题）

A. 对

B. 错

答案：A

解析：此题说法正确（选A）。在 MVC 架构中，Model（模型）充当观察目标（Subject）的角色。Model 管理应用程序的核心数据和业务逻辑，当数据发生变化时，Model 会通知所有注册的 View（视图/观察者），View 收到通知后获取最新数据并刷新显示。这种“数据变化→通知视图→自动更新界面”的机制是 MVC 架构的核心，也是观察者模式在实际框架中最经典的应用之一。

40. 观察者模式存在循环依赖时有可能导致系统崩溃。（）（判断题）

A. 对

B. 错

答案：A

解析：此题说法正确（选A）。在观察者模式中，如果出现循环依赖——A 观察 B 的同时 B 也观察 A——当 A 的状态发生变化时会通知 B 更新，B 的更新操作又可能修改 B 的状态，进而通知 A 更新，A 更新后又通知 B……如此往复形成死循环。这种无限递归调用最终会导致调用栈溢出（StackOverflowError）或系统无响应，严重时可能导致整个系统崩溃。因此在使用观察者模式时需要特别注意避免循环依赖的设计。

